Caso 3 – Concurrencia y Sincronización de Procesos

Andres Felipe Santana Callejas – 202311502

Daniel Esteban Galindo – 202414673

20/04/2026

1. Diseño del sistema (Diagrama de clases)

El sistema implementa una arquitectura concurrente tipo pipeline de procesamiento, donde múltiples hilos interactúan a través de estructuras compartidas como en este caso, los buzones, que sigue el patrón productor-consumidor.

Las clases principales son:

- **Sensor:** En este caso, es el productor de eventos
- **Broker:** Consumidor y redireccionador de eventos
- **Administrador:** Analizador de eventos sospechosos
- **Clasificador:** Enruta eventos a servidores
- **Servidor:** Consumidor final que procesa eventos
- **Buzon:** Estructura compartida sincronizada
- **Evento:** Unidad de información
- **Main:** Inicializa y coordina la ejecución

Relación entre clases

- Los sensores producen objetos "Evento" y los insertan en el buzón de entrada.
- El bróker consume eventos del buzón de entrada y los envía a:
    - Buzon de alertas, o
    - Buzon de clasificación
- El administrador consume eventos del buzón de alertas y:
    - Los descarta, o
    - Los reenvía al buzón de clasificación
- Los clasificadores consumen eventos del buzón de clasificación y los envían al buzón del servidor correspondiente.
- Cada servidor consume eventos de su buzón y los procesa.
- Todos los actores se comunican exclusivamente mediante instancias de la clase Buzón.

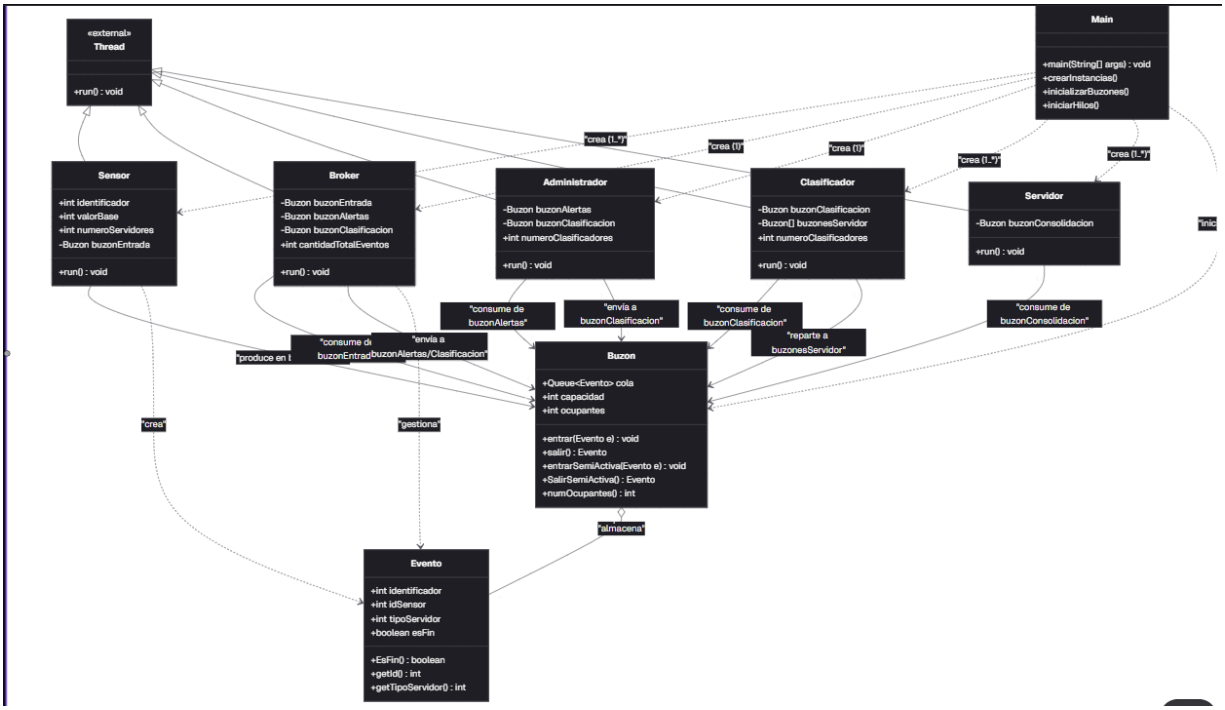
Sincronización

La clase Buzón implementa la sincronización utilizando:

- **Synchronized**
- **Wait()**
- **notifyAll()**
- **yield()**

Se implementan dos tipos de espera:

- Espera pasiva: Mediante wait() -> Bloquea el hilo
- Espera semi-activa: Mediante yield() -> Reintenta sin bloquear completamente)



2. Funcionamiento del programa

El sistema se ejecuta en las siguientes etapas:

1. Inicialización

El programa lee un archivo de configuración que define:

- Numero de sensores
- Numero de eventos base
- Numero de clasificadores
- Numero de servidores
- Capacidades de los buzones

A partir de estos valores se crean:

- Buzones compartidos
- Hilos para cada actor del sistema

2. **Generación de eventos (Sensores)**

Cada sensor:

- Genera base x idSensor eventos
- Deposita los eventos en el buzón de entrada
- Asigna:
    - Identificador único
    - Tipo de servidor (aleatorio)

3. **Procesamiento inicial (Broker)**

El broker:

- Consume eventos del buzón de entrada
- Genera un número aleatorio entre 0 y 200
- Clasifica:
    - Múltiplo de 8 -> evento anómalo -> buzón de alertas
    - Otro caso -> evento normal -> buzón de clasificación
- Al finalizar, envía un evento de fin al administrador

4. Análisis de alertas (Administrador)

El administrador:

- Consume eventos del buzón de alertas
- Genera un número entre 0 y 20
- Decide:
    - Múltiplo de 4 -> evento normal -> buzón de clasificación
    - Otro caso -> evento descartado
- Al recibir evento de fin:
    - Envía nc eventos de fin al buzón de clasificación

5. Clasificación de eventos (Clasificadores)

Cada clasificador:

- Consume eventos del buzón de clasificación
- Identifica el servidor destino
- Envía el evento al buzón correspondiente

Al recibir evento de fin:

- Termina su ejecución
- El último clasificador en finalizar:
    - Envía ns eventos de fin a los servidores

6. Procesamiento final (Servidores)

Cada servidor:

- Consume eventos de su buzón
- Simula procesamiento (100 ms – 1000 ms)
- Al recibir evento de fin:
    - Termina su ejecución

7. Terminación del sistema

El sistema finaliza cuando:

- Todos los hilos han terminado
- Todos los buzones están vacíos

3. Sincronización entre pareja de objetos

Sensor ↔ Buzón de entrada

- Tipo: productor–consumidor
- Mecanismo:
    - El sensor utiliza entrarSemiActiva()
    - Si el buzón está lleno, el hilo hace yield() (espera semi-activa)
- Sincronización:
    - Uso de synchronized para acceso exclusivo

    - notifyAll() para despertar consumidores

Broker ↔ Buzón de entrada

- Tipo: consumidor
- Mecanismo:
    - El broker utiliza salir ()
    - Si el buzón está vacío, el hilo entra en wait()
- Sincronización:
    - wait() → espera pasiva
    - notifyAll() al insertar nuevos eventos

Broker ↔ Buzones (alertas y clasificación)

- Tipo: productor
- Mecanismo:
    - Usa entrarSemiActiva()
- Sincronización:
    - Evita bloqueos activos innecesarios
    - Permite concurrencia controlada

Administrador ↔ Buzón de alertas

- Tipo: consumidor
- Mecanismo:
    - Uso de SalirSemiActiva()
- Sincronización:
    - Evita bloqueo completo
    - Reintento mediante yield()

Administrador ↔ Buzón de clasificación

- Tipo: productor
- Mecanismo:

    - Inserta eventos normales y eventos de fin
- Sincronización:
    - Uso de métodos sincronizados del buzón

Clasificadores ↔ Buzón de clasificación

- Tipo: consumidores concurrentes
- Mecanismo:
    - Uso de salir ()
- Sincronización:
    - Exclusión mutua mediante synchronized
    - Espera pasiva cuando no hay eventos

Clasificadores ↔ Buzones de servidores

- Tipo: productores
- Mecanismo:
    - Insertan eventos según el tipo de servidor
- Sincronización:
    - Acceso controlado con synchronized

Servidores ↔ Buzones de consolidación

- Tipo: consumidores
- Mecanismo:
    - Uso de salir()
- Sincronización:
    - Espera pasiva wait()
    - Procesamiento concurrente sin interferencia

4. Validación y pruebas

Para verificar el correcto funcionamiento del sistema se realizaron las siguientes pruebas:

Prueba 1: Ejecución Básica

Configuración:

- 3 sensores
- 2 eventos base
- 2 clasificadores
- 2 servidores

Resultados:

- Todos los eventos fueron procesados
- No hubo bloqueos
- Los buzones quedaron vacíos

Prueba 2: Flujo completo

Se verificó que:

- Los eventos pasan por todas las etapas:
    - Sensor -> broker -> administrador/clasificador -> servidor
- Los eventos anómalos son correctamente filtrados

Prueba 3: Terminación correcta

Se comprobó que:

- El broker envía evento de fin al administrador
- El administrador envía eventos de fin a clasificadores
- El último clasificador envía eventos de fin a servidores
- Todos los hilos terminan correctamente

```
Servidor: Evento con id200ha sido procesado
Servidor: Procesando el evento...
Servidor: Evento con id201ha sido procesado
Servidor: Procesando el evento...
Clasificador: Evento enviado al servidor número 1 con el id 304
Clasificador: Clasificador terminado por el evento fin con id -1
Último clasificador ? enviando eventos de fin a servidores
Servidor: Evento con id202ha sido procesado
Servidor: Procesando el evento...
Servidor: Evento con id302ha sido procesado
Servidor: Procesando el evento...
Servidor: Evento con id203ha sido procesado
Servidor terminado
Servidor: Evento con id303ha sido procesado
Servidor: Procesando el evento...
Servidor: Evento con id305ha sido procesado
Servidor: Procesando el evento...
Servidor: Evento con id304ha sido procesado
Servidor terminado
Sistema completado. Todos los hilos terminaron.
Numero de eventos en buzon de entrada: 0
Numero de eventos en buzon de clasificación: 0
Numero de eventos en buzon de Alertas: 0
PS C:\Users\andre\OneDrive\Documentos\GitHub\Caso_3\Sistema_IoT>
```

Prueba 4: Concurrencia

Se observó que:

- Múltiples hilos acceden a los buzones sin inconsistencias
- No se presentan condiciones de carrera
- No hay bloqueos mutuos (deadlocks)

Prueba 5: Integridad de los datos

Se verificó que:

- Todos los eventos generados son procesados o descartados correctamente
- No hay pérdida de eventos
- No se duplican eventos

Conclusión

El sistema implementa correctamente un modelo concurrente basado en productores y consumidores, utilizando mecanismos de sincronización adecuados. Se garantiza el flujo correcto de eventos, la detección de anomalías y la terminación controlada de todos los procesos sin inconsistencias ni bloqueos.